Dossier projet

Nom de naissance : Polard
Prénom : Corentin
Adresse : 30 rue de la Pradelle - 63000 Clermont-Ferrand

Titre professionnel visé : Développeur web et web mobile - niveau 5

Modalité d'accès :

- ☑ Parcours de formation
- ☐ Validation des Acquis de l'Expérience (VAE)

Organisme formateur : Simplon Clermont-Ferrand

Sommaire

Jeux Partage **4**
I - Introduction **4**
A - Contexte du projet 4
B - Public cible 4
C - Objectifs globaux 4
II - Conception **6**
A - UML 6
1 - Diagramme de cas d'utilisation 6
2 - Diagramme de classes 7
B - Charte graphique 7
1 - Couleurs 7
2 - Polices 8
3 - Logo 8
C - Maquettes 9
III - Outils et technologies **10**
A - Outils 10
1 - Organisation 10
2 - Gestion de versions 10
B - Base de données 11
C - Technologies et langages 11
D - Dépendances 12
E - APIs 12
F - Outil de build 12
IV - Backend **13**
A - Base de données 13
B - Les controllers 13
1 - L'authentification 14
2 - Un exemple de controller 15
3 - La méthode exécutée à l'appel de la route app_events 16
4 - La méthode exécutée à l'appel de la route app_create_event 18
5 - La méthode exécutée à l'appel de la route app_edit_event 19
V - Frontend **21**
A - Accessibilité, SEO et performance 21
1 - Accessibilité 21
2 - SEO 21
3 - Performance 22
B - CSS 22

C - JavaScript....22
1 - Vision globale du JavaScript dans le projet....22
2 - Map.js en détails....23
VI - Sécurité....25
VII - Tests....27
VIII - Déploiement....28
IX - Améliorations et perspectives d'évolution....29
A - Améliorations....29
1 - Messagerie en temps réel....29
2 - Système de notifications....29
3 - Sécurité et gestion des comptes....29
B - Perspectives d'évolution....29
1 - Un mini réseau social....29
2 - Amélioration de l'expérience ludique....30
X - Conclusion....31
XI - Annexes....32
A - UML....32
1 - Diagramme de cas d'utilisation....32
2 - Diagramme de classes....33
B - Schéma de la base de données....34
C - Lighthouse des pages principales....35
1 - La page "Les événements"....35
2 - La page d'un événement....35
3 - La page d'accueil....36

Jeux Partage

I - Introduction

A - Contexte du projet

Jeux Partage est un projet qui a pour objectif de **mettre en relation des personnes qui aiment les jeux de sociétés** afin d'**étendre leur cercle social** et de pouvoir **partager des moments conviviaux** autour des jeux de société.

L'idée de Jeux Partage m'est venue à partir de différentes situations rencontrées dans mon expérience de joueur. Amateur de jeux de société, il m'est arrivé à plusieurs reprises de ne pas pouvoir profiter de mes jeux préférés à cause du nombre minimum de participants nécessaires. Ce constat m'a fait réfléchir à la difficulté que peuvent rencontrer de nombreux joueurs lorsqu'ils souhaitent pratiquer leur passion.
D'un autre côté, j'ai toujours eu envie de découvrir Donjons et Dragons ou d'autres jeux de rôle similaires, mais je ne connaissais aucun groupe de joueurs pour tenter l'expérience.
Le **manque d'occasion et de contacts pour jouer** m'ont donné l'idée de Jeux Partage, un site qui permet aux amateurs et passionnés de trouver des partenaires de jeux.

B - Public cible

Le public cible est constitué de **toutes les personnes intéressées par les jeux de société**, qu'elles soient amatrices, passionnées, débutantes ou expertes. Ce public est varié en termes d'âge (des jeunes adultes aux personnes plus âgées) mais **partage un même besoin**: trouver facilement des partenaires de jeu près de chez eux et profiter de leur passion dans un cadre convivial et accessible.

C - Objectifs globaux

J'avais de nombreuses idées de fonctionnalités pour ce site. Néanmoins cela me paraissait un peu ambitieux de réaliser la totalité de ces fonctionnalités, tout en préparant les autres livrables dans le délai donné.
C'est pourquoi j'ai réfléchi à un **Produit Minimum Viable (MVP)** et établi une liste des fonctionnalités pour une première version de mon site :

- **Une création d'évènement** : permettant aux utilisateurs connectés d'organiser une séance pour jouer à des jeux avec d'autres utilisateurs. Un évènement contient des informations sur son titre, sa description, sa date, son adresse, ses jeux, son organisateur et ses participants, son nombre maximum de participants et si l'évènement est gratuit (par exemple si l'évènement se déroule dans un bar où une consommation est obligatoire).

- **Une liste d’évènements avec filtre par ville ou département** : recensant les évènements à venir proche de l’utilisateur avec pagination.

- **Un système d’inscription/désinscription aux évènement** : permettant à l’utilisateur connecté de s’inscrire ou de se désinscrire d’un évènement.

- **Une section contenant les évènements auxquels l'utilisateur connecté participe** : permettant à l’utilisateur de retrouver facilement les informations des évènements auxquels il est inscrit et de se désinscrire facilement ou de les modifier/supprimer si il en est l’organisateur.

- **Une page contenant toutes les informations détaillées de l’évènement, contenant une messagerie** : permettant aux utilisateurs de converser, d’obtenir des informations complémentaires.

- **Une gestion administrateur** : permettant à l’administrateur de gérer (Créer, Lire, Modifier, Supprimer) des jeux, des évènements, des messages, des catégories et de voir les informations (nom, prénom, évènements) des autres utilisateurs et de les supprimer.

II - Conception

A - UML

J’ai utilisé le logiciel **StarUML** pour réaliser les diagrammes de cas d’utilisation et de classes.

1 - Diagramme de cas d’utilisation

J’ai commencé par faire le diagramme de cas d'utilisation. Ce diagramme identifie les différents types d’utilisateurs du site, montre comment les utilisateurs interagissent avec le site, ce qu’ils peuvent faire ou ne pas faire et identifie les fonctionnalités principales.

J’ai identifié 3 acteurs différents :

- L’utilisateur non-connecté qui peut :
    - S’inscrire / Se connecter
    - Consulter la liste des jeux
    - Voir les détails des jeux
    - Gérer la liste des évènements avec possibilité de les filtrer
    - Consulter les pages statiques (page d’accueil, de mentions légales…)

- L’utilisateur connecté qui peut :
    - Réaliser les mêmes actions que l’utilisateur non-connecté (mis à part s’inscrire ou se connecter)
    - S’inscrire à un événement
    - Consulter la page “Mes évènements”
    - Voir les détails d’un évènement auquel il est inscrit
    - Envoyer un message sur la messagerie de l’évènement
    - Créer un événement
    - Modifier ou supprimer les évènements qu’il organise
    - Consulter son profil
    - Modifier son profil
    - Supprimer son compte et ses données
    - Se déconnecter

- L’administrateur qui peut :
    - Réaliser les mêmes actions que l’utilisateur connecté (hormis supprimer son compte)
    - Créer/Lire/Modifier/Supprimer des catégories, des jeux, des évènements
    - Lire/Supprimer des messages ou des utilisateurs

Vous pourrez retrouver le diagramme de cas d’utilisation en annexe.

2 - Diagramme de classes

Je me suis ensuite occupé du diagramme de classes. Celui-ci sert à montrer la structure de la base de données et du code, décrire les entités principales et leurs relations et aide à visualiser comment les données sont liées entre elles.

J'ai identifié six classes :

- **User :** représente un utilisateur avec son id et ses informations personnelles, son mot de passe, son email, son prénom, son nom, son rôle et éventuellement sa photo de profil.
- **Event :** représente un évènement créé par un utilisateur, avec des informations comme l'id, le titre, le lieu, la date, la description, la liste des participants, les jeux, si l'événement est gratuit, la durée prévue, le nombre maximum de participants et les informations de date de création et de modification de l'événement
- **Address :** représente une adresse d'un évènement et contient un id la rue, la ville, le code postal, la latitude et la longitude d'un événement
- **Game :** représente un jeu avec son id, son nom, sa description, une éventuelle image, les nombres minimum et maximum de joueurs ainsi que la durée moyenne d'une partie.
- **Category :** représente une catégorie de jeux de société et contient un id et un label.
- **Message** : contient les messages échangés entre utilisateurs contient le contenu du message et sa date de création.

Et les relations entre les classes :

- Un utilisateur peut créer plusieurs événements (OneToMany).
- Un utilisateur peut participer à plusieurs événements et un événement peut avoir plusieurs participants (ManyToMany).
- Un évènement peut concerner plusieurs jeux et un jeux peut-être dans plusieurs événements (ManyToMany).
- Un évènement à une adresse et une adresse à un évènement (OneToOne).
- Un jeu à une catégorie mais une catégorie peut-être liée à plusieurs jeux (OneToMany).
- Un utilisateur peut envoyer plusieurs messages mais un message est lié à un utilisateur (OneToMany).
- Un évènement peut-être lié à plusieurs messages mais un message est lié à un évènement (OneToMany).

Vous pourrez retrouver le diagramme de classes en annexe.

B - Charte graphique

J'ai cherché à définir une charte graphique afin de garantir une identité visuelle claire et agréable pour les utilisateurs tout en faisant attention au contraste des couleurs que je vérifiais sur le site dequeuniveristy.com et aux polices pour garantir une bonne accessibilité.

1 - Couleurs

Pour le site, les couleurs principales utilisées sont :

- #632024 : utilisé pour les fonds des headers, footers et boutons principaux. Cette couleur apporte un contraste fort et une ambiance chaleureuse.

- #F9F6E1 : couleur claire utilisée pour le fond principal du site et pour les textes des headers, footers et boutons, assurant lisibilité et confort visuel.

- #D5B893 : couleur utilisée pour les fonds des headers des cards des évènements, permettant de distinguer les sections tout en gardant une harmonie avec le reste du site.

- #25344F : couleur utilisée pour les titres, pour un contraste optimal et une hiérarchisation claire des informations.

- Noir (#000000) : couleur utilisée pour les textes principaux afin de garantir une lecture facile et nette.

J'ai longtemps hésité au cours du développement au sujet des couleurs, j'ai eu du mal à trouver des couleurs qui évoquent les jeux de sociétés et je n'ai pas trouvé de sites dans le même thème qui me plaisaient pour m'inspirer.
Au final j'ai gardé cette palette de couleurs qui crée une **atmosphère conviviale et chaleureuse**, tout en mettant en valeur les éléments importants comme les titres, boutons et sections interactives.

2 - Polices

Police des titres : **DM Serif Display**
Utilisée pour les titres et éléments de mise en avant.
Cette police avec empattement (serif) crée un contraste visuel avec la police sans-serif des paragraphes, ce qui améliore la hiérarchie et guide l'œil de l'utilisateur.
Adaptée à un thème ludique mais sérieux, elle donne au site une identité visuelle tout en restant lisible.

Police de base : Nunito, sans-serif
Utilisée pour les paragraphes, menus et textes secondaires.
Cette police sans-serif est moderne, lisible et douce. Elle transmet un sentiment de convivialité et de simplicité, adapté à l'univers des jeux de société où l'expérience utilisateur doit rester fluide et agréable.

3 - Logo

Mon logo est constitué du nom de mon site, et de 3 personnes réunies autour d'une table pour jouer aux cartes, le tout avec les couleurs choisies précédemment.



C - Maquettes

Après avoir trouvé les couleurs et les polices j'ai pu commencer à maquetter avec **Figma**. J'ai commencé par réaliser les maquettes pour mobiles que j'ai ensuite étendues et modifiées pour obtenir les maquettes au format desktop.
J'ai fait les maquettes en commençant par le header et par le footer.
Puis j'ai fait les maquettes de mes pages principales à savoir la page d'accueil, la page listant les évènements, une page avec le détail d'un évènement, le formulaire de création d'évènement, les pages d'inscription et de connexion.

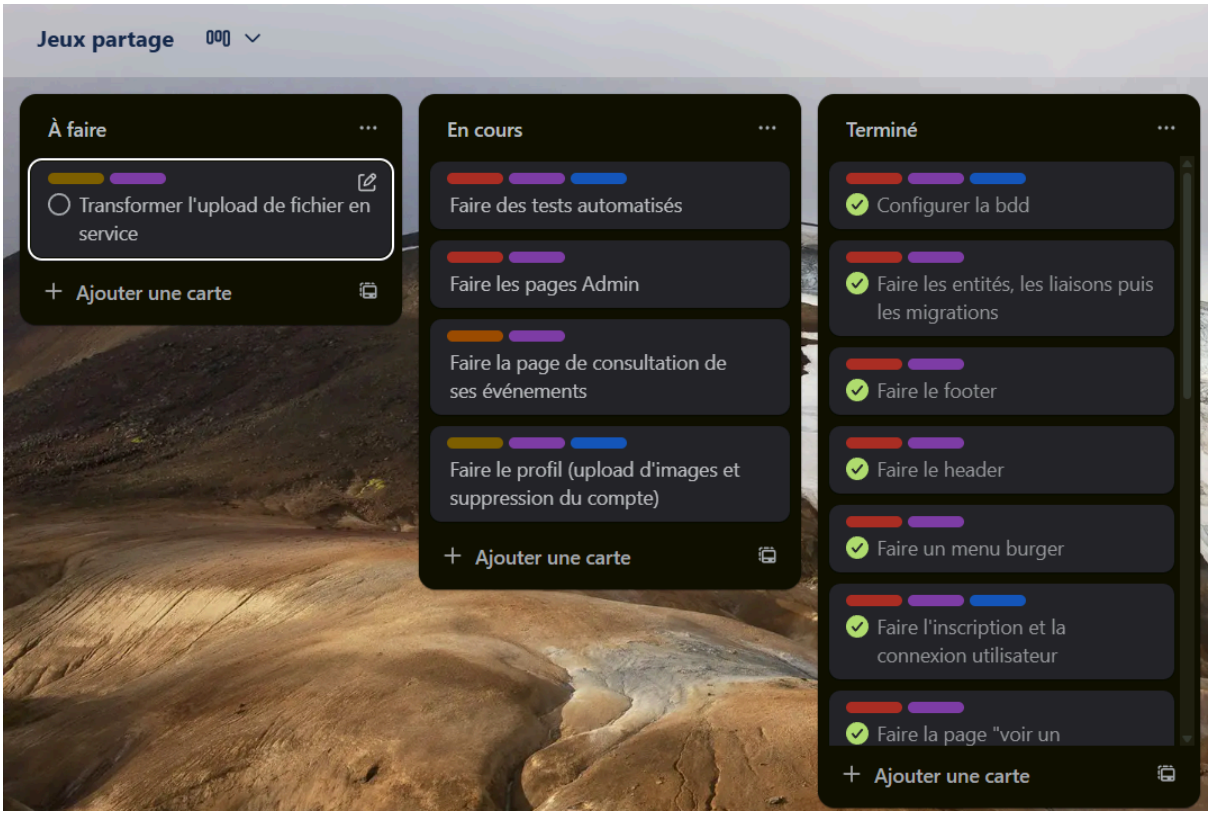
La maquette est accessible via ce lien :
https://www.figma.com/design/gqa0pDGjJa5LhyYT21OAcu/Jeux-Partage?node-id=0-1&p=f&t=RqYgFXKNxw7441Vp-0

III - Outils et technologies

A - Outils

1 - Organisation

J’ai utilisé **Trello** pour réaliser ce projet. Ça m’a permis de lister tout ce que j’avais à faire, de créer des checklists et de visualiser la quantité de travail qu’il me restait à faire. Pour organiser mon travail, j'ai utilisé un Trello de type KANBAN. Cette méthode m'a permis de structurer mes tâches de manière claire et efficace. J’ai structuré mon tableau en 3 parties : à faire, en cours, terminé.



2 - Gestion de versions

J’ai utilisé Git pour gérer le suivi des versions de mon projet **GitHub** pour le stocker en ligne. Cela me permet de revenir facilement à une version précédente en cas de problème.
Étant seul à travailler sur ce projet, j’ai effectué tous mes commits sur la branche principale, en les nommant de manière claire et explicite afin de pouvoir les retrouver facilement et suivre l’évolution du projet. J’ai utilisé principalement les commandes suivantes :

1. **git add** chemin/fichier : pour ajouter des fichiers à enregistrer dans le commit.
2. **git commit -m “Description du commit”** : pour enregistrer les modifications sur les fichiers ajoutés précédemment.
3. **git push** : pour envoyer le commit sur le dépôt GitHub.

La repository est accessible via ce lien : https://github.com/CorentinPolard/jeux-partage

B - Base de données

J'ai utilisé MySQL comme système de gestion de base de données relationnelle (SGBDR) pour mon projet. Il m'a permis de créer, stocker et organiser l'ensemble des tables indispensables au fonctionnement de l'application, telles que celles dédiées aux utilisateurs, aux évènements, aux inscriptions ou encore aux messages.
MySQL est particulièrement adapté car il est fiable, performant et largement utilisé dans le développement web, ce qui garantit une bonne maintenabilité.

C - Technologies et langages

Après avoir terminé la conception de la base de données avec les diagrammes UML, mis en place Trello pour la gestion de projet, et établi la maquette, j'ai commencé à structurer le backend du site avec **Symfony**, un **framework PHP**.

J'ai choisi Symfony 7.3.3 pour mon site (la dernière version stable) car nous avons utilisé ce framework à plusieurs reprises pendant la formation, et j'ai pu développer mes compétences en PHP en l'utilisant dans différents projets. Par ailleurs, Symfony est un framework PHP reconnu pour sa stabilité et sa maturité, avec une documentation complète et structurée qui facilite l'apprentissage et le développement.

Symfony repose sur une architecture **MVC (Model-View-Controller)**, ce qui permet de séparer clairement la logique métier, les données et l'affichage. Cette séparation améliore la lisibilité du code, facilite la maintenance et rend le débogage plus efficace.

Symfony s'appuie sur l'**ORM Doctrine**, qui simplifie grandement les interactions avec la base de données. Un ORM est un outil qui fait la **correspondance entre mon code et la base de données**. Au lieu de manipuler directement les tables et d'écrire des requêtes SQL à la main, je travaille avec des classes et des objets PHP, et Doctrine s'occupe de transformer ça en requêtes SQL adaptées qui modifient ou interagissent avec la base de données. Un des gros avantages, c'est que Doctrine génère des requêtes préparées. Ces **requêtes** sont plus **sécurisées**, car elles évitent par exemple les failles d'injection SQL.

En ce qui concerne le frontend, Symfony utilise **Twig**, un moteur de template, qui permet d'écrire du code **HTML5**, d'intégrer du **CSS3** et du **JavaScript**, mais avec des fonctionnalités supplémentaires qui simplifient le développement.
Par exemple, Twig permet d'**insérer des variables** directement dans le HTML, de faire des **boucles (for)** dans une page pour **éviter les répétitions** de code ou des **conditions (if)** pour afficher du code sous certaines conditions, de **réutiliser des morceaux de code** grâce à l'import de templates.
En résumé, Twig permet :

- De **mélanger facilement la logique de l'application Symfony avec le code HTML** tout en gardant une séparation propre entre le backend (PHP/Symfony) et le frontend (HTML/CSS/JS)
- D'**éviter les répétitions**

- De **garder un code plus clair et organisé**.

D - Dépendances

Pour améliorer l'expérience utilisateur, j'ai installé deux bibliothèques JavaScript avec npm dans mon projet :

Tom Select qui permet de transformer les champs de type select en éléments plus ergonomiques.
Tom Select offre des fonctionnalités comme la **recherche en temps réel**, la **sélection multiple** ou encore une meilleure lisibilité des options. Cela rend la saisie plus rapide et intuitive lors de la recherche de jeux de société à la création d'un évènement.

Leaflet qui permet d'**afficher et de manipuler des cartes interactives** dans l'application. Elle permet d'ajouter des marqueurs, de zoomer ou encore de naviguer facilement sur la carte. Leaflet est léger, performant et compatible avec différents fournisseurs de cartes comme OpenStreetMap. Dans le cadre de ce projet, elle a servi à enrichir l'interface en permettant aux utilisateurs de localiser et de visualiser les évènements directement sur une carte.

E - APIs

Pour enrichir la **fonctionnalité cartographique** de mon projet, j'ai utilisé Leaflet en combinaison avec des APIs de **géocodage** et **OpenStreetMap**. Cette combinaison permet aux utilisateurs de visualiser facilement l'emplacement des évènements et d'interagir avec la carte :

L'API de géocodage, trouvée sur le site adresse.data.gouv.fr, permet de **convertir une adresse ou un lieu en coordonnées géographiques** (latitude et longitude), afin de positionner précisément les évènements sur la carte.

OpenStreetMap fournit les données cartographiques (tuiles de carte) que Leaflet affiche et rend interactives pour l'utilisateur.

F - Outil de build

J'ai utilisé **Webpack** dans mon projet pour **regrouper et optimiser tous les fichiers front-end**, comme le JavaScript, le CSS et les images. Il **compile le code**, **gère les dépendances installées via npm** et **génère des fichiers prêts à être utilisés par le navigateur**, ce qui améliore les performances, facilite la maintenance et prépare le projet pour le déploiement.

IV - Backend

A - Base de données

Pour structurer la base de données, j'ai eu besoin de créer les **cinq entités** (classes) : **Event**, **Address**, **Game**, **Category** et **Message** en suivant le diagramme de classes.
Pour créer les entités j'utilise la commande de Symfony : **symfony console make:entity**
Cette commande nous permet de créer une entité ou d'en modifier une existante en choisissant son nom, ses attributs et ses relations avec d'autres entités.
La commande nous génère :

- L'entité : c'est la classe qui représente une table de la base de données, avec ses propriétés et ses relations.
- Le repository associé à l'entité : une classe qui sert d'intermédiaire entre le code et la base de données, contenant les requêtes de base pour récupérer les données (find, findAll, findBy et findOneBy).

Après avoir généré ces fichiers, j'ajoute des méthodes, si il en faut, aux entités.
Par exemple : j'ai créé la méthode isFull() dans l'entité Event qui compare le nombre maximum de participants avec le nombre de participants plus l'organisateur et renvoie true ou false.

```php
public function isFull(): bool
{
    return $this->getMaxNumberOfParticipants() === count($this->getParticipants()) + 1;
}
```

Une fois ces entités créées, je crée l'entité User avec la commande **symfony console make:user**. Cette commande fait la même chose que celle pour les entités classiques, sauf qu'en plus, elle applique automatiquement toutes les bonnes pratiques de sécurité liées aux utilisateurs.
Ensuite, je peux définir et gérer les relations entre elles (par exemple, game a une relation many to many avec event car un évènement peut contenir un ou plusieurs jeux et un jeux peut-être dans 0 ou plusieurs évènements).

Pour finir avec la base de données, j'utilise les commandes **symfony console make:migratio**n et **symfony console doctrine:migration:migrate**.
La première commande **génère un fichier de migration** (un plan) qui définit toutes les modifications à apporter à la base de données.
La seconde commande lit le fichier de migration et **applique les modifications** à la base de données. **Doctrine transforme mes classes PHP en tables SQL**.
Utiliser ces commandes permet de synchroniser mon code et ma base de données de façon propre, sans avoir à écrire toutes les requêtes SQL à la main. Ça me permet aussi de **garder un historique des évolutions de la base de données**.

Vous pourrez retrouver une représentation de la base de données en annexe.

B - Les controllers

Après avoir mis en place les modèles et les tables, j'ai continué avec la gestion des routes et la création des controllers.

Les controllers sont les classes qui **reçoivent les requêtes de l'utilisateur** (par ex. accéder à une page, envoyer un formulaire). Ils **utilisent le repository pour récupérer ou modifier les données** dans la base. Ensuite, ils **envoient les infos au frontend**. C'est ce qui fait le lien entre la Vue et le Modèle.
Les routes sont le lien entre les URL du site et les actions des controllers. Chaque fois qu'un utilisateur visite une page, la route correspondante indique quel controller doit traiter la requête. Les **routes peuvent aussi contenir des paramètres et servir à gérer la sécurité**, en limitant l'accès à certaines pages aux utilisateurs connectés ou à des rôles spécifiques. Ainsi, elles organisent la navigation et permettent au backend de répondre correctement à chaque action de l'utilisateur.

1 - L'authentification

J'ai commencé par créer le controller de la page d'accueil puis je me suis occupé de créer le système d'authentification (inscription/connexion) avec les commandes **symfony console make:auth** et **symfony console make:registration-form** qui génère automatiquement les fichiers nécessaires pour gérer la sécurité, comme le controller et le formulaire de connexion et d'inscription. J'ai modifié le registration-form pour ajouter des contraintes au mot de passe de l'utilisateur, qu'il fasse au minimum 12 caractères et qu'il contienne au moins une majuscule, une minuscule, un chiffre et un caractère spéciale conformément aux recommandations de la RGPD.

```
class RegistrationFormType extends AbstractType
    public function buildForm(FormBuilderInterface $builder, array $options): void
                ),
            ],
        ])
        ->add('plainPassword', RepeatedType::class, [
            'type' => PasswordType::class,
            'first_options'  => ['label' => 'Mot de passe'],
            'second_options' => ['label' => 'Confirmez le mot de passe'],
            'invalid_message' => 'Les mots de passe doivent correspondre.',
            'mapped' => false,
            'attr' => ['autocomplete' => 'new-password'],
            'constraints' => [
                new NotBlank(
                    message: 'Vous devez entrer un mot de passe.'
                ),
                new Length(
                    min: 12,
                    minMessage: 'Votre mot de passe doit contenir au moins {{ limit }} caractères',
                    max: 4096,
                ),
                new Regex(
                    pattern: '/[a-z]/',
                    message: 'Le mot de passe doit contenir au moins une minuscule.'
                ),
                new Regex(
                    pattern: '/[A-Z]/',
                    message: 'Le mot de passe doit contenir au moins une majuscule.'
                ),
                new Regex(
                    pattern: '/[0-9]/',
                    message: 'Le mot de passe doit contenir au moins un chiffre.'
                ),
                new Regex(
                    pattern: '/[^\w\s:]/',
                    message: 'Le mot de passe doit contenir au moins un caractère spécial.'
                )
```

Le système d'authentification de Symfony est **sécurisé** car il :

- **hache les mots de passe** pour qu'ils ne soient jamais stockés en clair

- **valide les formulaires** pour éviter les données incorrectes
- **gère les sessions et les rôles** pour contrôler l'accès aux pages selon les utilisateurs.

2 - Un exemple de controller

Après m'être occupé de l'authentification, je me suis occupé de l'EventController, le controller des évènements, qui sont le contenu principal de mon site. Je l'ai initié avec la commande **symfony console make:controller** qui crée le fichier contenant le controller et un fichier twig contenant un template de base.

J'ai initialisé mes routes, j'en ai 8 dans ce controllers : pour la liste d'évènements, voir les détails d'un évènement, créer, modifier et supprimer un évènement, pour qu'un utilisateur voit ses évènements, et pour s'inscrire et se désinscrire d'un évènement.
Exemple de quelques routes du controller :

```php
#[Route('/events')]
final class EventController extends AbstractController
{
    public function __construct(…
    ) {}

    #[Route('', name: 'app_events')]
    public function index(EventRepository $eventRepository, Request $request): Response
    {…
    }

    #[Route('/show/{id}', name: 'app_show_event', requirements: ['id' => '\d+'])]
    public function showEvent(Event $event, EntityManagerInterface $entityManager, Request $request): Response
    {…
    }
```

Ici on peut voir la route ayant pour url /events qui va récupérer tous des évènements dans le repository et les transmet à la vue.
Il y a aussi la route ayant pour url events/show/id qui prend en paramètre dynamique l'id d'un évènement et transfert les données liées à cet évènement à la vue.

3 - La méthode exécutée à l'appel de la route app_events

Méthode index() exécutée à l'appel de la route ayant pour url /events :

```php
#[Route('', name: 'app_events')]
public function index(EventRepository $eventRepository, Request $request): Response
{
    // Pagination
    $limit = 10;
    $page = $request->query->getInt('page', 1);

    // Filtre
    $city = $request->query->has('city') ? $request->query->get('city') : null;
    $departmentNumber = $request->query->has('departmentNumber') ? $request->query->get('departmentNumber') : null;

    // Formulaire de filtrage
    $form = $this->createForm(EventsFilterType::class, [
        'city' => $city,
        'departmentNumber' => $departmentNumber
    ]);
    $form->handleRequest($request);

    $filterBy = $form->isSubmitted() && $form->isValid() ? $form->getData() : [
        'city' => $city,
        'departmentNumber' => $departmentNumber
    ];

    if (in_array($filterBy['departmentNumber'], ["2A", "2B"])) {
        $filterBy['departmentNumber'] = 20;
    }

    // Récupération des évènements selon les filtres (si il y en a)
    $events = $eventRepository->paginate($page, $limit, "e", new DateTime(), 'eventAt', $filterBy['city'], $filterBy['departmentNumber']);

    // Calcul du nombre de pages pour la pagination
    $maxPages = $events->count() > 0 ? ceil($events->count() / $limit) : 1;
    if ($page < 1) {
        return $this->redirectToRoute('app_events', ['page' => 1]);
    }
    if ($page > $maxPages) {
        return $this->redirectToRoute('app_events', ['page' => $maxPages]);
    }

    return $this->render('event/events-list.html.twig', [
        'events' => $events,
        'maxPages' => $maxPages,
        'page' => $page,
        'form' => $form->createView(),
    ]);
}
```

Cette méthode index prend en paramètres une instance du **repository** lié à l'entité Event qui interagit avec la base de données et sert à accéder aux données des événements, ainsi qu'une une instance de la classe Request qui contient toutes les informations liées à la requête HTTP faite par l'utilisateur.

Dans cette partie du contrôleur, on s'occupe d'afficher la liste des événements de manière organisée et filtrable pour l'utilisateur grâce à :

- une **Pagination** : On définit un nombre maximum d'événements par page (10). On récupère le numéro de page actuel depuis l'URL ($page) et si aucun n'est spécifié, on prend la page 1.

  ```php
  $limit = 10;
  $page = $request->query->getInt('page', 1);
  ```

  On calcule ensuite le nombre total de pages nécessaires pour afficher tous les événements et on s'assure que l'utilisateur ne demande pas une page invalide.

  ```php
  $maxPages = $events->count() > 0 ? ceil($events->count() / $limit) : 1;
  if ($page < 1) {
      return $this->redirectToRoute('app_events', ['page' => 1]);
  }
  if ($page > $maxPages) {
      return $this->redirectToRoute('app_events', ['page' => $maxPages]);
  }
  ```

- un **Filtrage** (optionnel) : On récupère les valeurs des filtres envoyés par l'utilisateur via l'URL : ville (city) et numéro de département (departmentNumber) si elles existent.

```
$city = $request->query->has('city') ? $request->query->get('city') : null;
$departmentNumber = $request->query->has('departmentNumber') ? $request->query->get('departmentNumber') : null;
```

On crée un formulaire de filtrage (EventsFilterType) pour permettre à l'utilisateur de modifier ces filtres facilement. Le formulaire est déjà pré-rempli si il a déjà séléctionné des filtres.

```
$form = $this->createForm(EventsFilterType::class, [
    'city' => $city,
    'departmentNumber' => $departmentNumber
]);
```

Les valeurs sont ensuite validées et utilisées pour filtrer les événements.

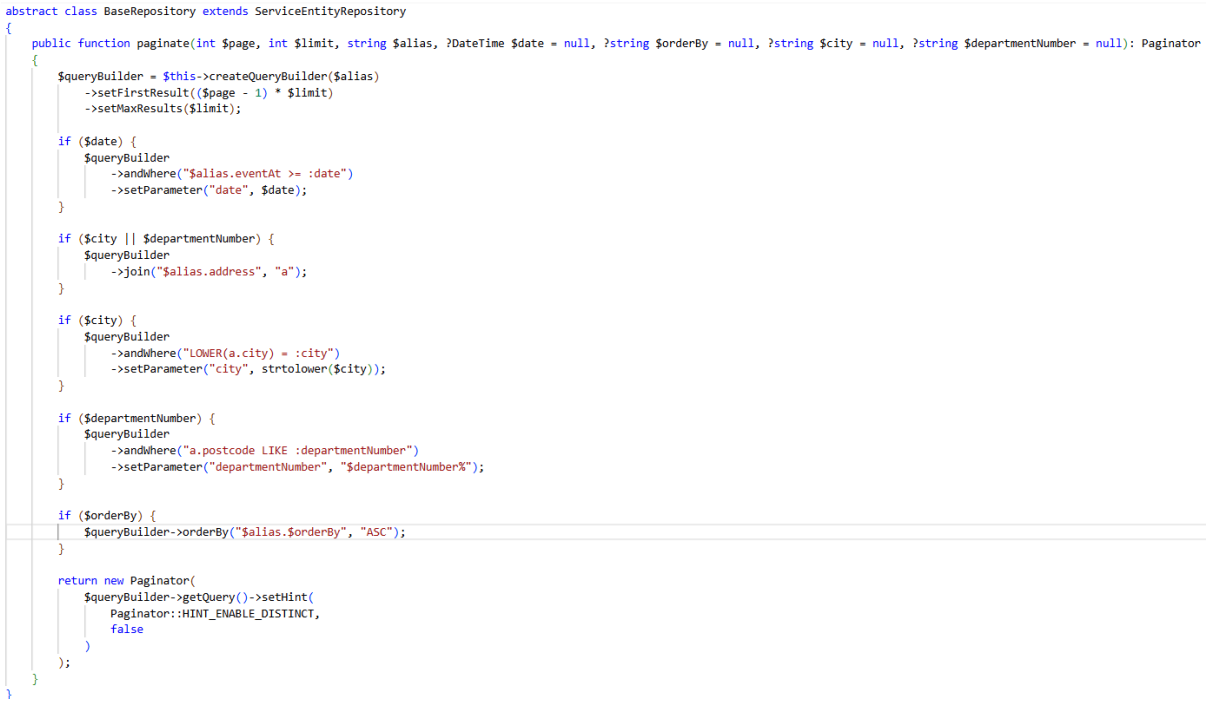
```
$filterBy = $form->isSubmitted() && $form->isValid() ? $form->getData() : [
    'city' => $city,
    'departmentNumber' => $departmentNumber
];

if (in_array($filterBy['departmentNumber'], ["2A", "2B"])) {
    $filterBy['departmentNumber'] = 20;
}
```

- **Récupération des événements** : Le EventRepository est utilisé pour chercher les événements en fonction de la page sélectionnée, de la date actuelle et des filtres s'il y en a.

```
$events = $eventRepository->paginate($page, $limit, "e", new DateTime(), 'eventAt', $filterBy['city'], $filterBy['departmentNumber']);
```

Pour ce faire, j'ai dû créer une classe BaseRepository dont EventRepository et GameRepository héritent les méthodes et y créer une méthode pour récupérer les données selon des conditions ou un ordre de tri si c'est indiqué.
Cette méthode renvoie un **Paginator** de Doctrine qui ne récupère que les événements de la page demandée ce qui permet d'afficher la liste par pages dans l'interface utilisateur :



```php
abstract class BaseRepository extends ServiceEntityRepository
{
    public function paginate(int $page, int $limit, string $alias, ?DateTime $date = null, ?string $orderBy = null, ?string $city = null, ?string $departmentNumber = null): Paginator
    {
        $queryBuilder = $this->createQueryBuilder($alias)
            ->setFirstResult(($page - 1) * $limit)
            ->setMaxResults($limit);

        if ($date) {
            $queryBuilder
                ->andWhere("$alias.eventAt >= :date")
                ->setParameter("date", $date);
        }

        if ($city || $departmentNumber) {
            $queryBuilder
                ->join("$alias.address", "a");
        }

        if ($city) {
            $queryBuilder
                ->andWhere("LOWER(a.city) = :city")
                ->setParameter("city", strtolower($city));
        }

        if ($departmentNumber) {
            $queryBuilder
                ->andWhere("a.postcode LIKE :departmentNumber")
                ->setParameter("departmentNumber", "$departmentNumber%");
        }

        if ($orderBy) {
            $queryBuilder->orderBy("$alias.$orderBy", "ASC");
        }

        return new Paginator(
            $queryBuilder->getQuery()->setHint(
                Paginator::HINT_ENABLE_DISTINCT,
                false
            )
        );
    }
}
```

La méthode appelée par la route renvoie ensuite la liste des événements, le formulaire pour les filtres et les informations de pagination à la vue pour être affichés à l'utilisateur de manière intuitive.

4 - La méthode exécutée à l'appel de la route app_create_event

```php
#[Route('/create', name: 'app_create_event')]
public function createEvent(EntityManagerInterface $entityManager, Request $request): Response
{
    $event = new Event();

    $form = $this->createForm(EventType::class, $event);
    $form->handleRequest($request);


    if ($form->isSubmitted() && $form->isValid()) {
        $event->setOrganizer($this->getUser());

        $entityManager->persist($event);
        $entityManager->flush();

        return $this->redirectToRoute('app_events');
    }

    return $this->render('event/create-event.html.twig', [
        'form' => $form->createView(),
    ]);
}
```

Cette méthode permet à l'utilisateur de créer un évènement via un formulaire et de le stocker en base de données. En même temps que de créer les méthodes dans les controllers, je générais les formulaires dont j'avais besoin avec la commande **symfony console make:form** et je les complétais pour configurer les champs nécessaires (les types d'input, les labels, les contraintes…).

Au final la méthode createEvent :

1. Instancie un nouvel objet Event.
2. Crée un formulaire basé sur la classe EventType lié à cet objet.
3. Gère la soumission du formulaire avec la requête HTTP :
   Si le formulaire est soumis et valide :
   Elle assigne l'utilisateur courant comme organisateur de l'évènement.
   Elle enregistre l'évènement en base de données avec l'EntityManager.
   Elle redirige vers la liste des évènements.
   Si le formulaire n'est pas soumis ou invalide, elle affiche la vue du formulaire de création.

$entityManager->persist($event); prépare l'objet $event à être inséré dans la base de données.
$entityManager->flush(); : exécute réellement l'opération en base de données : l'évènement est inséré (INSERT) dans la table correspondante.

En SQL cela correspondrait à faire :
INSERT INTO event (title, description, organizer_id, ...) VALUES ("Titre de l'évènement", "description de l'évènement", id, …);

5 - La méthode exécutée à l'appel de la route app_edit_event

```php
#[Route('/edit/{id}', name: 'app_edit_event')]
public function editEvent(Event $event, EntityManagerInterface $entityManager, Request $request): Response
{
    if ($event->getOrganizer() === $this->getUser()) {

        $form = $this->createForm(EventType::class, $event);
        $form->handleRequest($request);


        if ($form->isSubmitted() && $form->isValid()) {
            $event->setUpdatedAt(new DateTime());
            $entityManager->flush();
            return $this->redirectToRoute('app_show_event', ['id' => $event->getId()]);
        }

        return $this->render('event/edit-event.html.twig', [
            'event' => $event,
            'form' => $form->createView(),
        ]);
    } else {
        $this->addFlash('error', "Vous n'êtes pas autorisé à modifier cet évènement.");
        return $this->redirectToRoute('app_events');
    }
}
```

La méthode editEvent permet à l'organisateur d'un événement de modifier ses informations via un formulaire. Cette méthode récupère l'évènement ayant pour id la valeur

du paramètre id dans l'url (par exemple /events/edit/24 récupèrera l'événement ayant pour id 24 si il existe) :

1. Vérifie que l'utilisateur courant est bien l'organisateur, si oui, le formulaire est affiché et traité. Sinon, il est redirigé avec un message d'erreur.
2. Crée un formulaire Symfony basé sur la classe EventType, lié à l'évènement en paramètre de la méthode. Cela permet d'afficher un formulaire pré-rempli avec les données de l'événement à modifier.
3. Si le formulaire est soumis et valide :
   La date de mise à jour de l'événement est actualisée
   Les modifications sont enregistrées en base de données (requête SQL de type UPDATE).

En SQL cela correspondrait à faire :

```
UPDATE event
SET
champ1 = valeur1,
champ2 = valeur2
WHERE id = eventId;
```

V - Frontend

La partie front-end de mon projet a pour objectif de rendre l'application ergonomique, interactive et agréable à utiliser pour toutes les tailles d'écrans. Elle permet aux utilisateurs de naviguer facilement, de filtrer et de consulter les événements, ainsi que d'interagir avec la carte des événements.
Twig utilise les **blocs** pour permettre de créer des **templates réutilisables et modulaires**. Les templates principaux (base.html.twig et admin-base.html.twig) définissent des blocs comme le titre ou le contenu, et chaque template enfant peut remplacer ou compléter ces blocs selon ses besoins. Cela permet de réutiliser le code commun, de simplifier la maintenance et de garder une structure claire pour toutes les pages.

J'ai commencé par développer le template base.html.twig qui servira de **base pour toutes mes pages côté utilisateurs**. Ce template définit la **structure HTML** commune à toutes les pages publiques. Il inclut dans le **head** les **métadonnées**, le **titre** (personnalisable via un bloc Twig), le **favicon**, et les **fichiers CSS et JS** générés par Webpack Encore.
Il contient aussi le **header** qui contient le logo, le menu burger pour mobile et une navigation adaptée selon le rôle de l'utilisateur. Le **bloc main** affiche les messages flash puis le contenu spécifique à chaque page. Le **footer** propose les liens légaux et le copyright. Toutes les pages publiques héritent de ce template et injectent leur contenu dans le bloc main.
Je me suis ensuite occupé du côté administrateur. Toutes les pages côtés administrateur sont définies par le template admin-base.html.twig.
Le côté administrateur et le côté public ont des bases différentes car le template admin est pensé pour la gestion et l'efficacité tandis que le côté public doit avoir une navigation plus agréable et un design plus agréable.

A - Accessibilité, SEO et performance

1 - Accessibilité

Les **images** importantes contiennent des attributs **alt** pour être compréhensibles par les technologies d'assistance.
La **charte graphique** prend en compte le **contraste des couleurs** pour que les textes soient lisibles par tous.
Les polices choisies (Nunito pour le texte et DM Serif Display pour les titres) assurent une **lecture facile et agréable**.
Les **formulaires sont correctement étiquetés**, et la **navigation peut se faire entièrement au clavier**, respectant les bonnes pratiques d'accessibilité.
Le site est conçu en **mobile-first**, en partant du mobile afin de faciliter la responsivité. Le design se concentre sur la clarté, la simplicité et la lisibilité, ce qui rend le site plus accessible à tous les utilisateurs.

2 - SEO

Chaque page possède des titres pertinents pour améliorer son référencement ainsi qu'une méta description.

Les **images sont optimisées** avec WebP et des tailles adaptées pour accélérer le chargement.
La **structure sémantique des pages** (titres <h1>, <h2> etc., balises <main> <section> et <nav>) permet aux moteurs de recherche de comprendre facilement le contenu.

3 - Performance

Les **images ont été compressées et converties en WebP avec Squoosh**, ce qui réduit leur poids tout en conservant une bonne qualité.
L'utilisation de **Webpack Encore** permet de regrouper et **optimiser les fichiers JS et CSS**.

Vous pourrez retrouver le résultat des lighthouse des pages principales : liste des événements, détail d'un événement et page d'accueil en annexe.
Lighthouse est un outil open source développé par Google qui permet d'auditer et d'analyser la qualité des pages web. Il est principalement utilisé par les développeurs et les équipes web pour améliorer les performances, l'accessibilité et l'expérience utilisateur d'un site.

B - CSS

En ce qui concerne le css, j'ai plusieurs fichiers css pour faciliter la maintenance :

- **variables.css** : contient des variables CSS qui définissent les couleurs et les polices ainsi que les imports de police. Utiliser des variables permet de changer les polices ou les couleurs dans toutes les autres feuilles de style en modifiant uniquement quelques lignes de code.
- **common.css** : regroupe les styles globaux utilisés sur le côté utilisateurs et le côté administrateur (layout, boutons, formulaires, modals, etc.).
- **header.css** : contient le style du header côté utilisateurs. J'ai décidé de le séparer car il contient beaucoup de css pour le menu burger.
- **style.css** : contient du css uniquement utilisé sur le côté utilisateur.
- **app.css** : importe variables.css, header.css, style.css pour le côté utilisateurs.
- **admin-style.css** : contient du css uniquement utilisé sur le côté administrateur.
- **admin.css** : importe variables.css et admin-style.css (pour l'admin).

Les fichiers app.css et admin.css sont importés respectivement dans les fichiers base.html.twig et admin-base.html.twig avec webpack. Ces fichiers servent de bases pour tous les autres templates du site.

C - JavaScript

1 - Vision globale du JavaScript dans le projet

Dans le projet, le JavaScript joue un rôle essentiel pour améliorer l'interactivité et l'expérience utilisateur sur le site. Le JavaScript du projet sert à dynamiser l'interface, à sécuriser certaines actions, et à offrir des fonctionnalités avancées qui ne seraient pas possibles uniquement avec HTML et CSS. Il contribue à rendre l'interface plus fluide, sans rechargement complet de page, et à offrir une expérience utilisateur moderne.
Voici une vue d'ensemble des fichiers et de leur utilité :

- **burgerMenu.js** gère des fonctionnalités dynamiques côté client, comme l'ouverture/fermeture du menu burger.
- **checkPassword.js** gère la vérification des mots de passe lors de l'inscription
- **deleteModal.js** et **deleteAccount.js** gère la gestion des modals de suppression.
- **searchGames.js** recherche de jeux en temps réel.
- **geocodage.js** permet de récupérer des coordonnées gps à partir d'une adresse fournie dans le formulaire de création d'événements grâce à l'API geocodage.
- **map.js** permet d'intégrer des éléments interactifs comme l'affichage de cartes grâce a Leaflet et aux coordonnées gps.

2 - Map.js en détails

Le script map.js permet d'afficher automatiquement une carte interactive centrée sur une adresse donnée, avec un marqueur, en s'appuyant sur Leaflet et les coordonnées gps fournies :

Pour commencer je récupère les coordonnées grâce au controller. Cette instruction utilise le Serializer de Symfony pour convertir l'objet $event en JSON, en ne gardant que les données marquées dans le groupe "coordinates" (ici latitude et longitude). Cela permet d'envoyer uniquement les informations utiles au JavaScript.

```
$jsonCoordinates = $this->serializer->serialize(
    $event,
    'json',
    [
        'groups' => ['coordinates'],
    ]
);
```

Si l'événement possède bien des coordonnées GPS (latitude et longitude). Je les transmet au twig, dans une div, dans un attribut data-address qui stocke les coordonnées en JSON.

```
{% if event.address.latitude and event.address.longitude %}
<div id="map" class="maps" data-address="{{ jsonCoordinates }}">
</div>
{% endif %}
```

Enfin, si il y a la div contenant les coordonnées, je les récupère dans le fichier map.js, je les convertit en un objet js et je les stocke dans une variable.
La carte est alors créée avec L.map() dans la div qui contient les coordonnées puis elle est centrée sur ces coordonnées avec un zoom de 16.
Les tuiles de fond sont importées par le service OpenStreetMap, ajoutées grâce à L.tileLayer().
Enfin, un marqueur est placé sur la carte à l'endroit des coordonnées pour localiser précisément l'adresse.
Une capture d'écran du code est disponible à la page suivante.

map.js

assets > scripts > map.js > ...

```
1  import "leaflet/dist/leaflet.css";
2  import L from "leaflet";
3  import markerIcon from 'leaflet/dist/images/marker-icon.png';
4  import markerIcon2x from 'leaflet/dist/images/marker-icon-2x.png';
5  import markerShadow from 'leaflet/dist/images/marker-shadow.png';
6
7  L.Icon.Default.mergeOptions({
8      iconUrl: markerIcon,
9      iconRetinaUrl: markerIcon2x,
10     shadowUrl: markerShadow,
11 });
12
13 let mapContainer = document.querySelector("div[data-address]");
14 if (mapContainer) {
15     const coordinates = JSON.parse(mapContainer.dataset.address).address;
16
17     mapContainer = L.map('map').setView([coordinates.latitude, coordinates.longitude], 16);
18     L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
19         attribution: '&copy; <a href="https://www.openstreetmap.org/copyright">OpenStreetMap</a>',
20         maxZoom: 19
21     }).addTo(mapContainer);
22
23     L.marker([coordinates.latitude, coordinates.longitude]).addTo(mapContainer);
24 }
```

Si les coordonnés ne sont pas présentés, la carte n'est tout simplement pas affiché pour l'utilisateur.

VI - Sécurité

J'utilise la dernière version stable de Symfony (7.3.3). Symfony rend la sécurité du site plus simple et robuste grâce aux commandes automatiques comme **symfony console make:user**, **symfony console make:auth** ou **symfony console make:registration-form**. Ces outils créent automatiquement les classes et formulaires nécessaires pour gérer les utilisateurs, la connexion, l'inscription. Grâce à ces générateurs, les bonnes pratiques de sécurité (**hachage des mots de passe via l'algorithme le plus efficace, protection CSRF, gestion des rôles et des accès**) sont appliquées automatiquement, ce qui limite les erreurs et garantit que les données des utilisateurs restent protégées.
Par ailleurs, les données saisies par les utilisateurs sont validées côté serveur, même si des contrôles existent côté front-end pour le mot de passe à l'inscription, afin de s'assurer qu'elles respectent les règles attendues.

Le fichier **security.yml** permet de réserver des routes à certains utilisateurs. En effet, un système de rôles permet de distinguer les utilisateurs non authentifiés, les simples utilisateurs et les administrateurs.
Par exemple, les utilisateurs non connectés auront accès uniquement à la page d'accueil, les pages "légales" accessibles via le footer, la liste des événements et les pages des jeux (liste et informations).
Les utilisateurs connectés auront en plus accès à toutes les pages qui concernent les événements (la page de détails si il est inscrit) et au profil.
L'administrateur aura lui accès, en plus, au backoffice.

```
access_control:
    - { path: ^/admin, roles: ROLE_ADMIN }
    - { path: ^/profile, roles: ROLE_USER }
    - { path: ^/events/, roles: ROLE_USER }
```

Dans les controllers j'autorise certaines actions comme la modification et la suppression d'instances uniquement si l'utilisateur en est l'auteur :
Par exemple, un utilisateur ne peut pas supprimer un évènement dont il n'est pas l'auteur.

```
#[Route('/delete/{id}', name: 'app_delete_event', requirements: ['id' => '\d+'])]
public function deleteEvent(Event $event, EntityManagerInterface $entityManager): Response
{
    if ($event->getOrganizer() === $this->getUser()) {
        $entityManager->remove($event);
        $entityManager->flush();
    } else {
        $this->addFlash('error', "Vous n'êtes pas autorisé à supprimer cet évènement.");
    }

    return $this->redirectToRoute('app_events');
}
```

Lorsqu'il y a un problème, le controller envoie un flash clair à la vue qui aide l'utilisateur à identifier l'erreur sans exposer d'informations sensibles.

Autre exemple, un utilisateur ne peut pas s'inscrire à un événement si il en est l'auteur, si il y est déjà inscrit ou si il est plein :

```php
#[Route('/register/{id}', name: 'app_register_event', requirements: ['id' => '\d+'])]
public function register(Event $event, EntityManagerInterface $entityManager): Response
{
    $user = $this->getUser();

    if (
        $event->getOrganizer() !== $user
        && !$event->getParticipants()->contains($user)
        && !$event->isFull()
    ) {
        $event->addParticipant($user);
        $entityManager->persist($event);
        $entityManager->flush();

        return $this->redirectToRoute('app_show_event', ['id' => $event->getId()]);
    } else {
        $this->addFlash('error', "Vous ne pouvez pas vous inscrire à cet évènement. Il est complet ou vous êtes déjà inscrit.");
        return $this->redirectToRoute('app_events');
    }
}
```

VII - Tests

Après avoir développé une première version fonctionnelle de mon site, j'ai développé des tests automatisés avec phpUnit.
Les tests informatiques permettent de **détecter les bugs**, d'**améliorer la qualité du code** et de **garantir la fiabilité de l'application**. Ils réduisent les coûts de développement et de maintenance en corrigeant les défauts en amont et assurent que les fonctionnalités fonctionnent correctement pour les utilisateurs.

J'ai commencé par configurer une base de données de test avec les mêmes tables et relations que dans la vraie base de données afin de ne pas altérer les vraies données si je veux tester mon code. J'ai ensuite généré des fixtures pour alimenter cette base et écrit des tests.

Voulant me concentrer sur les fonctionnalités du site, je n'ai pas pris le temps de réaliser tous les tests. Cependant, j'ai effectué des tests fonctionnels vérifiant l'accès aux pages administrateur pour les différents utilisateurs (non-connecté, connecté et admin) sur le formulaire d'inscription pour vérifier le succès ou non en fonction du mot de passe entré. Et un test unitaire qui vérifie que la fonction getFullName de la classe User fonctionne correctement.
La réalisation de tests automatisés complets aurait été préférable, car elle aurait garanti une vérification systématique de toutes les fonctionnalités, facilité la détection rapide des bugs futurs et assuré une meilleure stabilité et maintenabilité du projet sur le long terme.

Voici un exemple de test qui vérifie qu'un utilisateur ayant le rôle admin puisse accéder aux backoffice :

```php
public function testAdminAccessAdmin(): void
{
    $client = static::createClient();
    $entityManager = $client->getContainer()->get('doctrine')->getManager();

    $user = $entityManager->getRepository(User::class)->findOneBy(['email' => 'admin@example.com']);
    $client->loginUser($user);

    $client->request('GET', '/admin');

    $this->assertSelectorTextContains('h1', 'menu administrateur');
}
```

```
Runtime:       PHP 8.3.14
Configuration: C:\Users\coren\Documents\Simplon\final\jeux-partage\phpunit.xml

.......                                                             7 / 7 (100%)

Time: 00:40.386, Memory: 58.00 MB

OK (7 tests, 12 assertions)
```

VIII - Déploiement

Pour déployer le site, plusieurs étapes sont nécessaires :

1. **Choisir un hébergeur** compatible PHP8.3.14/MySQL (par exemple OVH ou o2switch) et un serveur.
2. Trouver/louer un **nom de domaine**.
3. **Transférer les fichiers** sur le serveur via **FTP**.
4. **Configurer le serveur web** pour que la racine du site pointe vers le dossier public/.
5. Installer les dépendances PHP sur le serveur avec la commande :
   composer install --no-dev --optimize-autoloader
   L'option –no-dev exclut les dépendances utiles uniquement au développement comme phpUnit par exemple qui sert pour les tests.
   L'option –optimize-autoloader rend le projet plus rapide en production en organisant mieux la façon dont PHP charge les classes
6. **Créer, configurer la base de données** et mettre à jour le fichier .env avec les identifiants corrects.
7. **Compiler les assets** en production avec Webpack :
   npm install
   npm run build
8. **Sécuriser (http → https)** : Le site doit être accessible en HTTPS pour protéger les données échangées entre l'utilisateur et le serveur. Cela se fait en installant un certificat SSL/TLS

Je n'ai pas encore déployé le site en ligne pour plusieurs raisons. D'abord, le projet est encore en phase de développement et certaines fonctionnalités méritent d'être consolidées avant une mise en production. J'ai choisi de me concentrer sur la conception et la mise en place des principales fonctionnalités plutôt que sur la configuration d'un serveur d'hébergement. De plus, je n'ai pas encore comparé et choisi d'hébergeur.

IX - Améliorations et perspectives d’évolution

A - Améliorations

1 - Messagerie en temps réel

Actuellement, la messagerie passe par un formulaire Symfony classique, ce qui oblige à recharger la page à chaque envoi de message ou pour voir s' il y a des nouveaux messages.
L’utilisation d’un système de **WebSocket** permettrait d’avoir une messagerie instantanée, plus fluide et proche d’une expérience type « chat ».

2 - Système de notifications

Notifier les utilisateurs lorsqu’ils reçoivent un message, lorsqu’un événement approche ou lorsqu’une demande d’inscription est validée.
Cela pourrait être fait via des notifications internes (icône cloche, pop-up dans le site) en temps réel grâce aux websockets ou par email pour rappeler aux utilisateurs les événements proches.

3 - Sécurité et gestion des comptes

Mettre en place une **validation des emails** lors de l’inscription pour **s’assurer de l’identité et de la confiance des utilisateurs**. Lorsqu’un utilisateur s’inscrit, Symfony peut **envoyer un email de confirmation**. Cet email contient un **lien unique avec un token**. Tant que l’utilisateur n’a pas cliqué sur ce lien, son compte reste inactif. Une fois validé, un champ isVerified (par exemple) dans la base passe à true et l’utilisateur peut se connecter.

Ajouter une fonctionnalité de **réinitialisation du mot de passe** en cas d’oubli. De façon très similaire à la validation des emails, les utilisateurs recevraient un mail avec un lien sécurisé et temporaire contenant un token les amenant sur une page leur permettant de réinitialiser leur mot de passe.

Permettre aux utilisateurs de **modifier leur photo de profil** pour personnaliser leur compte.

B - Perspectives d’évolution

1 - Un mini réseau social

Il serait possible de transformer le site en véritable réseau social autour des jeux de société en ajoutant plusieurs fonctionnalités qui faciliterait l’utilisation du site pour les utilisateurs :

- A**jouter des amis** afin de retrouver facilement les personnes avec qui on a passé un bon moment et voir leurs événements en priorité. Ce serait possible en créant une nouvelle table en base de données (par exemple friendship) qui gère les relations entre utilisateurs.
- Cette table contiendrait deux utilisateurs liés par une demande d’ami validée.

- **Système de likes** sur les événements. Il suffirait d'ajouter une autre table intermédiaire entre user et événement afin d'enregistrer les likes des utilisateurs sur les événements.
- **Liste de jeux préférés** partagée entre utilisateurs.

2 - Amélioration de l'expérience ludique

Afficher les jeux les plus populaires en fonction du nombre de like par les utilisateurs. Afficher les jeux les plus populaires aiderait les nouveaux utilisateurs à découvrir rapidement les jeux les plus tendances et cela valoriserait les jeux qui rassemblent le plus de joueurs et donne des idées aux organisateurs d'événements.

Mettre en place un système de vote pour choisir collectivement les jeux d'un événement. Lors de la création d'un événement, l'organisateur pourrait proposer une liste de jeux et les participants voteraient pour leur jeu préféré directement sur la page de l'événement. Le jeu qui recevrait le plus de votes serait sélectionné automatiquement, ou affiché comme choix recommandé.
Les **joueurs se sentent impliqués** dans le choix, ce qui renforce la convivialité et l'aspect collaboratif.

Ajouter une bibliothèque de jeux personnelle où chaque utilisateur pourrait gérer et partager sa collection. Cela serait possible en ajoutant, sur la page qui recense les jeux, une option permettant à chaque utilisateur d'ajouter un jeu à sa collection personnelle. Concrètement, une table supplémentaire dans la base de données pourrait gérer la relation entre les utilisateurs et les jeux. Cela permet de mieux connaître les goûts des joueurs et de faciliter la mise en relation entre personnes qui partagent des jeux en commun.

X - Conclusion

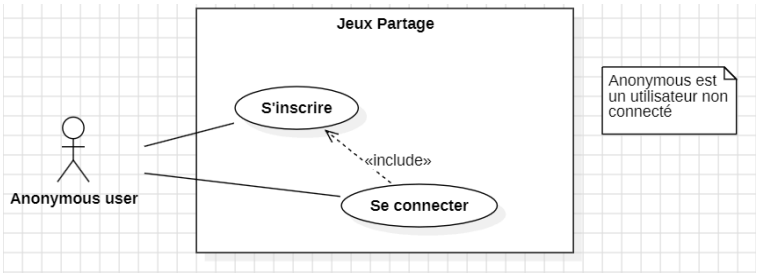
Jeux Partage m’a permis de mettre en pratique mes connaissances en développement web avec Symfony en créant un site concret qui répond à un besoin. Même si certaines fonctionnalités comme la messagerie en temps réel ou les notifications n’ont pas encore été développées, le site reste solide et évolutif. Les bonnes pratiques de sécurité, de performance et d’accessibilité ont été appliquées dès le départ pour garantir une expérience fiable et agréable aux utilisateurs.
Enfin, ce projet m’a donné une vision claire des améliorations possibles et des perspectives d’évolution : validation des emails, réinitialisation du mot de passe, système d’amis, votes et classement des jeux, bibliothèque de jeux, etc. Ces évolutions permettraient de transformer le site en une véritable plateforme interactive et sociale.

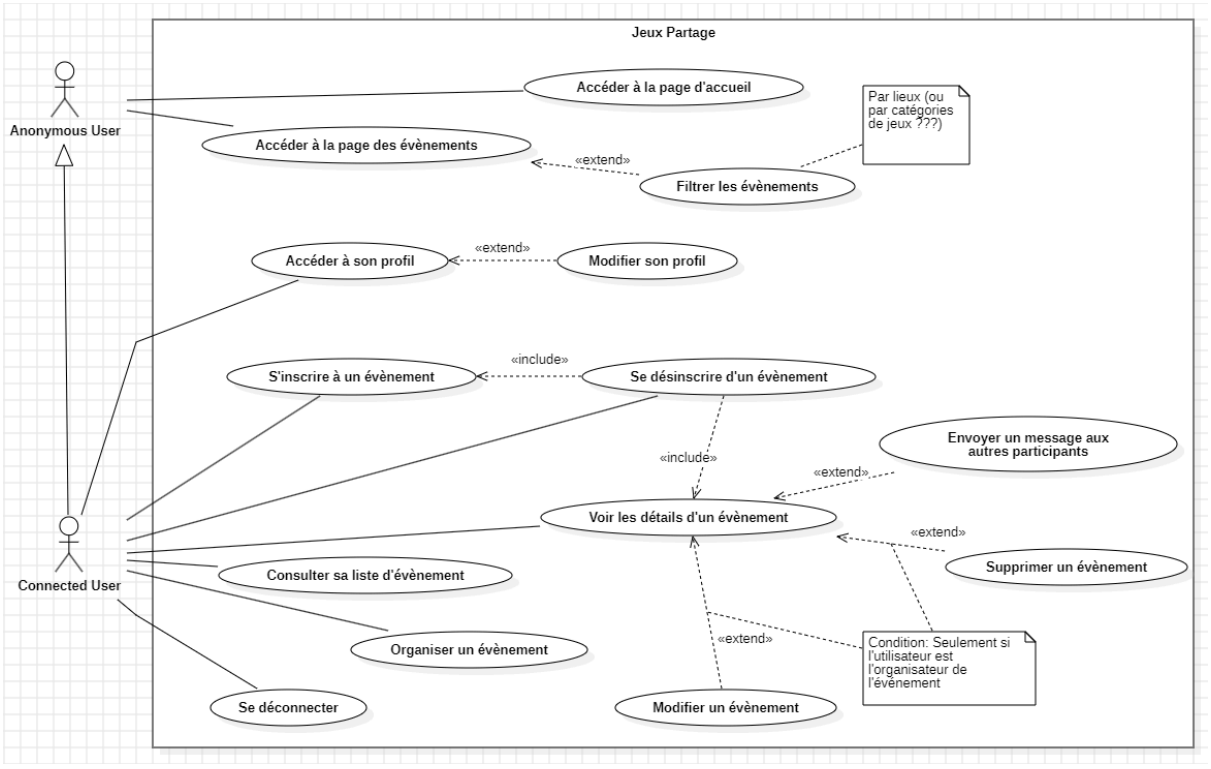
En résumé, ce projet a été une expérience formatrice, me permettant de consolider mes compétences techniques, d’appréhender les bonnes pratiques du développement web et de réfléchir aux besoins réels des utilisateurs pour construire un produit évolutif et sécurisé. J’ai hâte de poursuivre son développement après la formation pour enrichir ses fonctionnalités et améliorer encore l’expérience utilisateur.
Je suis heureux de m’être lancé dans le développement et je tiens à remercier Simplon ainsi que toute l’équipe pédagogique qui m’ont accompagné et permis de progresser tout au long de la formation. À l'issue de ma formation, j’espère avoir l’opportunité de continuer dans cette voie, trouver un emploi et continuer à monter en compétences.

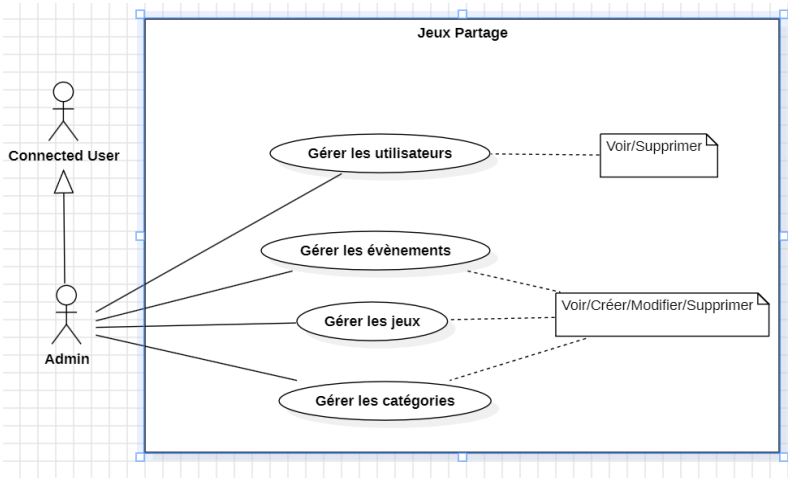
XI - Annexes

A - UML

1 - Diagramme de cas d'utilisation

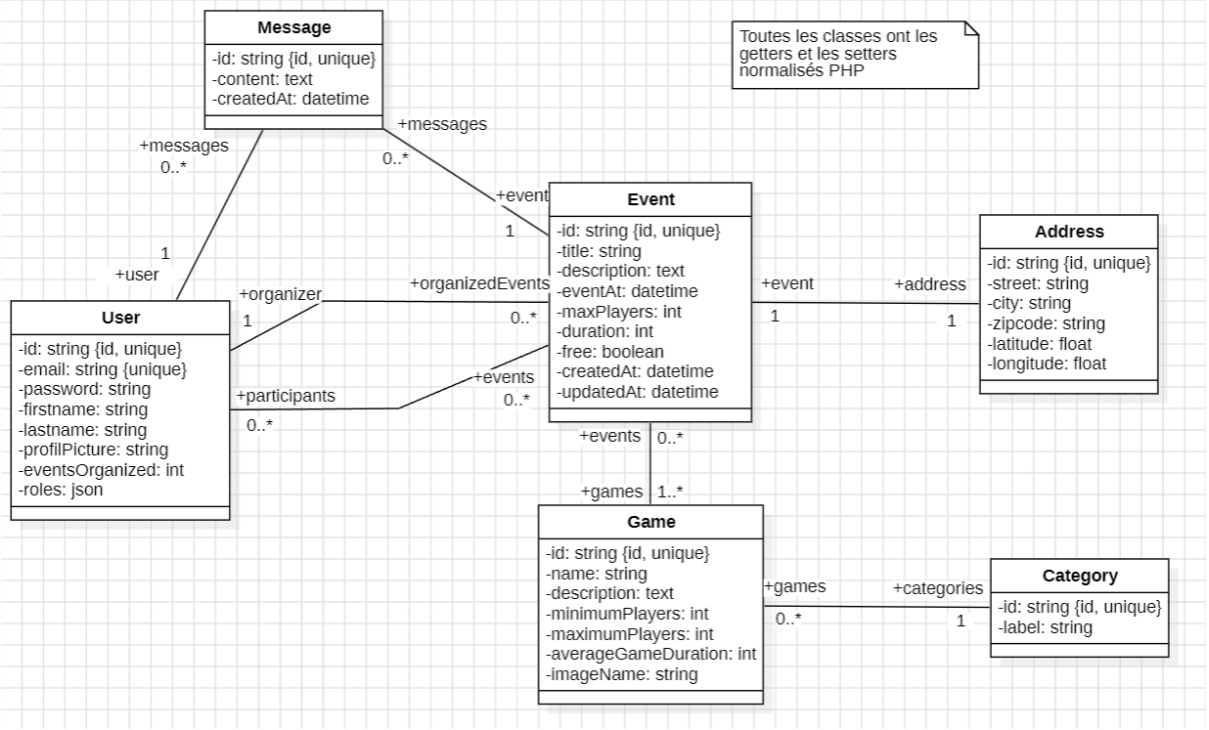






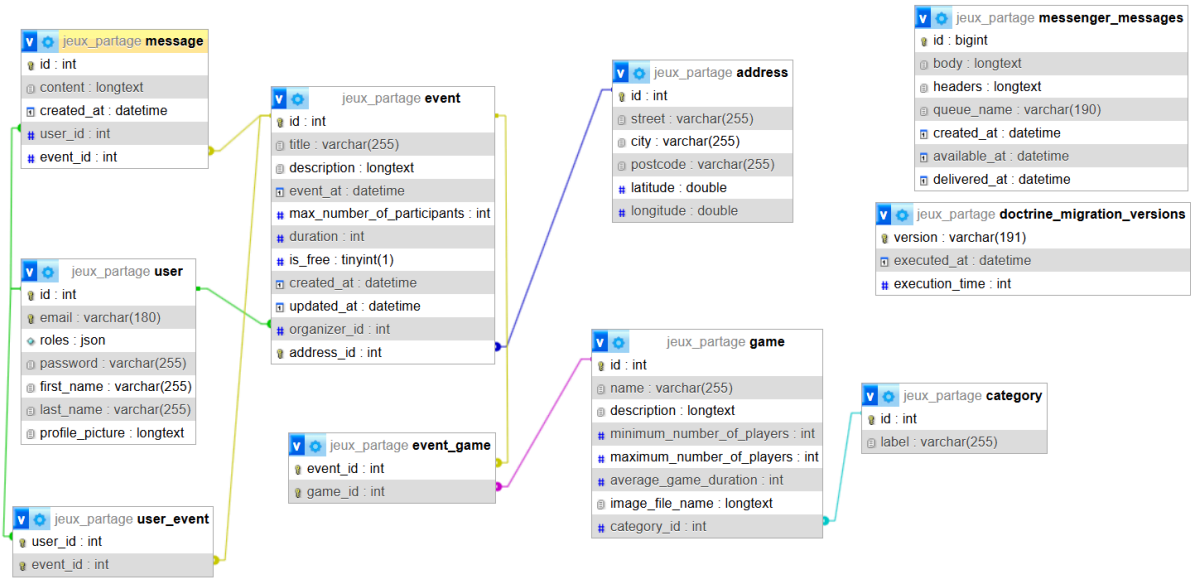
[Retourner à la section II.A.1 - Diagramme de cas d'utilisation]

2 - Diagramme de classes



[Retourner à la section II.A.2 - Diagramme de classes]

B - Schéma de la base de données



On peut voir deux tables dans la base de données qui ont été généré par Symfony :
messenger_messages
Cette table est créée automatiquement par Symfony Messenger. Elle sert à stocker les messages envoyés dans le système de messagerie asynchrone (même si je ne l'utilise pas encore pour l'instant). Chaque message correspond à une tâche à exécuter en arrière-plan, comme envoyer un email, traiter des données ou gérer une file d'attente. Tant que le message n'est pas traité, il reste dans cette table.

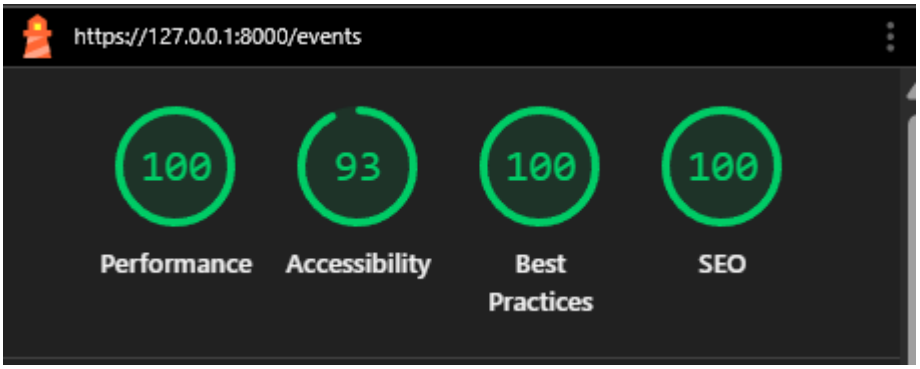
doctrine_migration_versions
Cette table est utilisée par Doctrine Migrations pour suivre les migrations de la base de données. Chaque fois qu'on ajoute une nouvelle migration (par exemple pour créer une table, modifier une colonne ou ajouter un index), Doctrine enregistre le numéro de version dans cette table. Cela permet de savoir quelles migrations ont déjà été appliquées et d'éviter de les exécuter deux fois.

[Retourner à la section IV.A - Base de données]

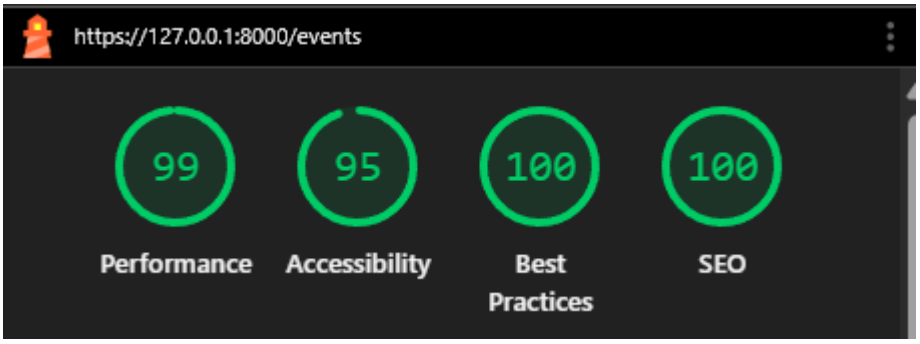
C - Lighthouse des pages principales

1 - La page “Les événements”

Version dekstop :

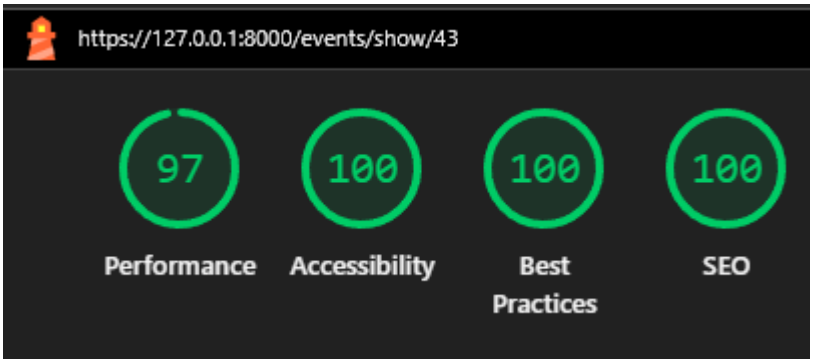


Version mobile :

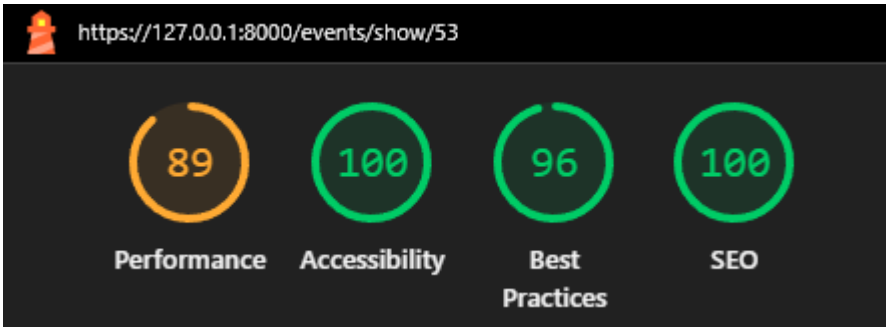


2 - La page d’un événement

Version desktop :

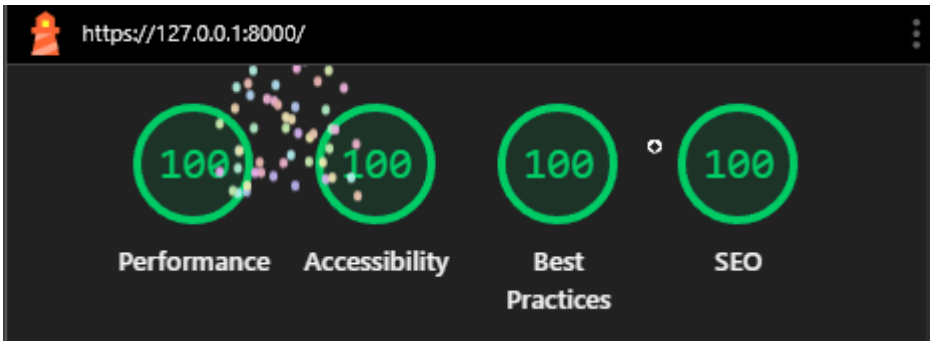


Version mobile :

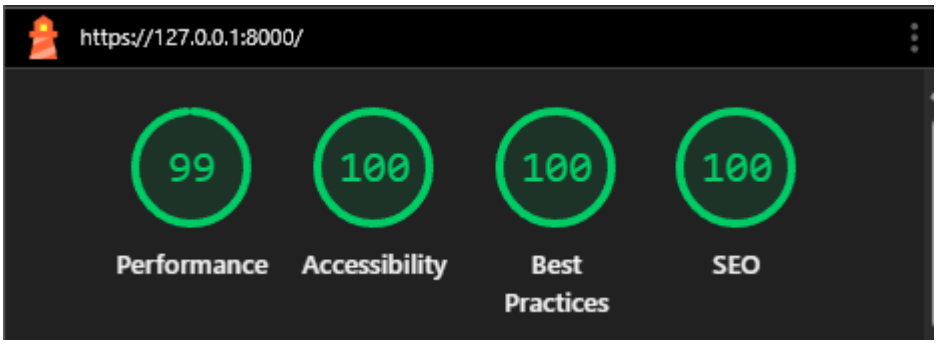


3 - La page d’accueil

Version desktop :



Version mobile :



Retourner à la section V.A.3 - Performance